

DEVELOPER BUILD SPEC · V1

Build Pointili — the web app.

A café loyalty + games platform. This spec is everything a developer needs to build the **app** (client hub, games, owner dashboard, caisse) on a **Supabase** backend. It excludes the marketing landing page.

Next.js + Supabase

Postgres · Auth · Storage · RLS

 Mobile view ONLY Owner-controlled

No landing page

00 · THE GOAL

What we're building & why

Pointili turns one-time café customers into regulars — by rewarding them for coming back. The whole product is one loop, and it **starts and ends with consuming** (buying). Games and rewards are the fun in the middle that pulls people back to buy again.

The loop (memorise this — the entire app serves it)

 **Consume** →  **Earn points** →  **Play &**  **Redeem** →  **Return** →  **Consume...**

Points are earned by **buying** (credited by staff at the caisse), **never** by playing. Playing the wheel gives prize codes; points buy real rewards. The developer's job is to make this loop **fun, useful, and nice** for the client — and **fully configurable by the owner**.

01 · SCOPE

What to build (and not)

In scope

- **Client web app** (the diner): auth, loyalty card, games, reward store.

Out of scope

- **Marketing landing page** — do not build.

- **Owner web app:** dashboard, caisse (credit/redeem), and the full control panel.
- **Staff caisse** access (PIN).
- **Supabase backend:** schema, RLS, auth, storage, server-authoritative RPCs.
- Super-admin basics (create cafés, toggle features) — optional for v1.

- **Desktop / tablet layouts** — see the mobile-only mandate below.
- QR menu / ordering / POS — Pointili is games + loyalty only.
- Native mobile apps — this is a web app (opened by scanning a QR).

 MOBILE-ONLY MANDATE — read carefully (this saves ~40% of the work)

Build **only the mobile view** (design at 375–430px width). The client reaches the app by scanning a QR on their phone; the owner manages from their phone too.

Do not build any desktop or tablet layout.

On wider screens, just center the phone-width app in a `max-w-[480px]` column — no `md:` / `lg:` breakpoints, no responsive grids. One layout, phone-sized, everywhere.

02 · CONTENTS

Report map

- 03

Tech stack & architecture
- 04

Roles & auth
- 05

Supabase data model
- 06

Security: RLS + server RPCs
- 07

The client app (screens)
- 08

The owner app (screens)
- 09

Owner control panel — every knob
- 10

Moment-by-moment build spec
- 11

The 3 make-or-break improvements
- 12

Build order (phases)
- 13

Definition of done

03 · ARCHITECTURE

Tech stack

LAYER	CHOICE	NOTES
Frontend	Next.js (App Router) + React + TypeScript	Server components for reads, client components for interactive hub/games.

Styling	Tailwind CSS	Mobile-only. Design tokens for the brand (see design system).
Backend	Supabase	Postgres + Auth + Storage + Row-Level Security + Edge/RPC.
Privileged writes	Next.js API routes w/ Supabase service-role + Postgres RPC functions	All value-affecting mutations (points, spins, redemptions) run server-side.
Auth	Supabase Auth (owners) · custom phone+PIN (diners) · PIN (staff)	See §04.
Storage	Supabase Storage (public reward-images bucket)	Reward & logo images.
Hosting	Vercel (frontend) + Supabase cloud	Env vars: NEXT_PUBLIC_SUPABASE_URL , *_ANON_KEY , SERVICE_ROLE_KEY .

The golden rule of the backend

The client never decides anything that has value. Balances, prize outcomes, cooldowns, and redemptions are computed by

Postgres RPCs

or service-role routes. The browser only *requests* — the server *decides*. This is the anti-cheat foundation.

04 · ROLES

Who uses it & how they log in

ROLE	AUTH	WHAT THEY DO
Diner (client)	Phone number + 4-digit PIN (custom, NOT Supabase Auth). One global identity across all cafés. Session = signed token in localStorage.	Earn points, play, redeem, track their card. Onboards in ~10s, no email, no app.
Owner	Supabase Auth (email + password).	Manages one café: dashboard, caisse, and the control panel.
Staff	Owner enables a PIN; staff act inside the owner's café.	Run the caisse: credit points, validate reward codes.

Super-admin

Supabase Auth + `role='super_admin'` in `profiles` .

Create cafés, toggle features platform-wide. (v1: minimal.)

Why phone+PIN for diners

Zero friction is the whole game. No email verification, no app install, no password. Scan QR → enter phone → 4-digit PIN → done. First visit grants the welcome bonus instantly (see §11).

05 · BACKEND

Supabase data model

Postgres tables. Every café is a tenant (`businesses`); all café-scoped data carries `business_id` . Points are an append-only **ledger** — a balance is `sum(delta)` , never a mutable column (auditable, race-safe).

TABLE	KEY COLUMNS	PURPOSE
<code>profiles</code>	id→auth.users, role, email	Owner / super-admin accounts.
<code>businesses</code>	id, owner_id, name, slug UNIQUE , status, primary_color, logo_url, design_settings jsonb	The café (tenant). <code>design_settings</code> holds most owner toggles.
<code>accounts</code> (diners)	phone (global), pin_hash, name	Global diner identity across cafés.
<code>loyalty_programs</code>	business_id PK, active, points_per_tnd , welcome_points , redeem_expiry_hours	Per-café points config.
<code>loyalty_rewards</code>	id, business_id, label, points_cost , image_url, active, position	The reward catalogue (the boutique).
<code>points_ledger</code>	id, business_id, customer_phone, delta , reason, created_at	Source of truth for balances. +earn / –redeem / welcome / adjust.
<code>loyalty_redemptions</code>	id, business_id, phone, reward_id, code , status, expires_at	A pending/claimed reward code, validated at the counter.
<code>games</code>	id, business_id, type, active, config jsonb	The wheel/slot. <code>config</code> = mode, odds, cooldown, gates, slot settings.
<code>prizes</code>	id, game_id, label, position, active	Wheel segments. Weights/isLose live in <code>games.config.prizeConfig</code> .

<code>plays</code>	id, business_id, phone, device, game_id, created_at	Play log — powers cooldown/rate-limit.
<code>wins</code>	id, business_id, phone, prize_id, code , status, expires_at	A won prize code, claimed at the counter.
<code>diner_streaks</code>	phone, current, longest, xp, freeze, play-counters	Streaks + XP/levels + achievements source (global per phone).
<code>diner_cafes</code>	phone, business_id, first_played_at	"Passport" — cafés a diner has visited.
<code>cafe_events</code>	id, business_id, kind, multiplier, starts_at, ends_at	Owner-scheduled boosts (double points/XP).
<code>app_staff</code>	business_id, pin, caps	Staff caisse identities.

Balance query (the one to memorise)

```
select coalesce(sum(delta),0) from points_ledger where business_id=$1 and customer_phone=$2
```

— points are stored by
(café, phone)
, so a diner can be credited before they even sign up (points wait for them).

06 · SECURITY

RLS + server-authoritative RPCs

Row-Level Security (RLS) — on for every table

- **Owners** read/write only rows where `business_id ∈ (their businesses)` .
- **Public/anon** may read only `active/paused` cafés + their public reward list (for the client hub).
- **Super-admin** policy grants full access where `profiles.role='super_admin'` .
- **Service role** bypasses RLS — used only inside trusted server routes/RPCs.

Server-authoritative RPCs (Postgres functions, `security definer`)

These are the only way value changes. The client calls a route → the route calls the RPC → the RPC decides & writes atomically.

```
-- Consume → Earn: staff credits a phone at the caisse
credit_points(business_id, phone, amount_tnd) returns balance
-- writes +points to points_ledger (points_per_tnd × amount), grants welcome bonus once
```

```
-- Play: server picks the prize, enforces cooldown, everyone-wins
play_game(slug, phone, device) returns { prizeIndex, code, nextPlayAt }
  -- validates cooldown from plays; picks prize by config weights; writes plays + wins
  -- NEVER trust prize/odds from the client

-- Redeem: atomic spend
redeem_at_counter(business_id, phone, reward_id) returns code
  -- checks balance ≥ cost, writes −points + a redemption code, all-or-nothing

-- Engagement
touch_diner_streak(phone, xp) returns streak -- consecutive-day count + XP
diner_wallet(phone) returns cards -- all cafés with activity
```

Non-negotiable

If the browser can set its own points or pick its own prize, the product is broken. Cooldowns, odds, balances, and redemptions are **server-side only**.

07 · CLIENT APP

The diner experience (mobile)

Reached at `pointili.app/<café-slug>` by scanning the café's QR. One space, persistent bottom nav: **Ma carte · Jeux · Boutique**.

SCREEN	JOB	KEY ELEMENTS
Onboarding / Auth	Convert a scan into an account in ~10s.	Value-first intro (points + spin + rewards) → phone → 4-digit PIN → welcome bonus lands.
Ma carte	The home: status + the "almost there" nudge.	Points balance, progress to next reward, streak, level (secondary), active reward codes.
Jeux	The fun: play the wheel/slot.	Anticipation build-up → everyone-wins reveal → celebration (confetti + receipt).
Boutique	Spend points on real rewards.	Reward ladder (cheap→aspirational), tap to redeem → code to show at counter.

Win / Receipt

Make winning feel like winning.

Printed-receipt style prize, code + expiry, "add to my card".

08 · OWNER APP

The owner experience (mobile)

SCREEN	JOB
Dashboard	Proof it works: repeat customers, points issued, rewards redeemed, plays — the return-rate trend.
Caisse	The core daily action: enter a phone (or scan the diner's QR) → credit points for a purchase; validate a reward/win code.
Rewards	Manage the boutique catalogue (label, cost, image, active, order).
Games	Configure the wheel/slot: prizes, odds, cooldown, gates.
Events	Schedule boosts (double points/XP for a weekend).
Settings	Brand (logo, name, colour), loyalty on/off, points rate, welcome bonus, expiry.

09 · OWNER CONTROL

Every knob the owner controls

This is a hard requirement: **every engagement mechanic is owner-configurable**. Each control below maps to a column or a `design_settings` / `games.config` key. Ship sensible defaults so a café works out-of-the-box, but expose all of it.

CONTROL	WHERE IT LIVES	DEFAULT
Loyalty on / off	<code>design_settings.loyaltyEnabled</code>	on
Points per dinar	<code>loyalty_programs.points_per_tnd</code>	1
Welcome bonus (on + amount)	<code>loyalty_programs.welcome_points</code>	10
Reward catalogue + prices	<code>loyalty_rewards</code> rows	3 seeded

Reward-code expiry (hrs)	<code>loyalty_programs.redeem_expiry_hours</code>	48
Wheel on / off	<code>games.active</code>	off
Slot machine on / off	<code>games.config.slotEnabled</code>	off
Prizes + win odds (everyone-wins)	<code>prizes</code> + <code>games.config.prizeConfig</code> (weights, isLose)	—
Play frequency (cooldown hrs)	<code>games.config.cooldownHours</code>	24
"Scan to play" presence gate	<code>games.config.qrGate</code>	off
Play gates (survey/follow to unlock)	<code>games.config.gates</code>	off
Events (double points/XP)	<code>cafe_events</code> rows	none
Points expiry (on + months)	<code>design_settings.pointsExpiryMonths</code> new	off
Streaks / levels shown	<code>design_settings.showEngagement</code> new	on
Brand: logo / name / colour	<code>businesses.logo_url</code> / <code>name</code> / <code>primary_color</code>	—

Design principle for the panel

Group controls by moment (Earn / Play / Reward / Return), not by table. The owner thinks "how often can they spin?", not "which jsonb key". Every control needs a plain-French label + one-line helper.

10 · THE CORE

Moment-by-moment build spec

The loop has four moments. Each one has a bar to clear, the client experience, the owner controls, the backend, and a "done when". **Build the app to serve these four moments.**



1 · Consume → Earn

AT THE CAISSE

THE BAR

Points land **instantly and visibly**, zero friction.

CLIENT SEES	A "+12 points" pop with a sound; balance grows; first-ever visit shows the welcome bonus landing.
OWNER CONTROLS	points per dinarwelcome bonus on/amountpoints expiry
BACKEND	<code>credit_points()</code> RPC via a staff caisse route; writes <code>points_ledger</code> ; grants welcome bonus once per (café, phone).
DONE WHEN	Staff credits a phone in <5s and the diner's balance + "+points" animation update live.



2 · The Spin

THE FUN CORE

THE BAR	Feels like play, not gambling — and never "you lost".
CLIENT SEES	Real anticipation (slow-down + tick), an everyone-wins reveal, then a celebration (confetti + printed receipt). One free spin per visit/day.
OWNER CONTROLS	wheel on/offslot on/offprizes + oddscooldownscan-to-play gateunlock gates
BACKEND	<code>play_game()</code> RPC — server picks the prize by owner weights, enforces cooldown from <code>plays</code> , writes <code>wins</code> . Odds never trusted from client.
DONE WHEN	Every spin returns a prize, respects the owner's cooldown, and can't be re-triggered by the browser.



3 · The Reward

THE PAYOFF

THE BAR	Rewards are desirable AND reachable (the #1 thing people get wrong).
CLIENT SEES	A ladder (cheap→aspirational), and always " just 40 pts to your free coffee ". Redeem → code to show at counter.
OWNER CONTROLS	reward cataloguepoint costsimagescode expiry

BACKEND

`redeem_at_counter()` RPC — checks balance, writes `-points + a code` atomically. Staff validates the code in the caisse.

DONE WHEN

A diner with enough points redeems, gets a code, and staff can mark it used — once.



4 · The Return

THE WHOLE POINT

THE BAR

A reason to come back that's **worth the trip** — without nagging.

CLIENT SEES

Streaks ("keep your 7-day streak 🔥"), "**your spin is ready**", weekend **events**, an almost-full reward they won't abandon.

OWNER CONTROLS

`events (double pts)` `streaks shown` `cooldown = spin cadence`

BACKEND

`touch_diner_streak()` on each play; `cafe_events` multiplier applied in `credit_points / play_game`.

DONE WHEN

A returning diner sees their streak advance and any active event boost applied automatically.

11 · MAKE OR BREAK

The 3 things — each owner-controlled

1 Instant first value

On visit #1, the diner gets **welcome points + a free spin** before doing anything. Never make them wait to see the point.

`owner controls` welcome bonus on/off + amount · whether visit-1 grants a free spin.

2 Always "almost there"

Show progress to the next reward **everywhere** ("40 pts to a free coffee"). This goal-gradient nudge drives more returns than anything else. Computed from balance vs the cheapest reward the diner

can't yet afford.

owner controls the reward ladder & prices define how near "almost there" feels — this is the tuning lever.

3 Everyone wins + celebrate

No losing feeling — the wheel always gives something — and every win/milestone gets a real moment (confetti, receipt, "+points").

owner controls the prize set + odds (incl. small "consolation" prizes) so "everyone wins" stays true and affordable.

The "nice" trap — enforce simplicity

The app already has points, levels, XP, achievements, passport, leaderboard, events.

Do not show all of it at once.

Core loop stays dead simple —

points → spin → reward

. Levels/passport/leaderboard are quiet secondary layers (behind a tap), and the owner can hide them ([showEngagement](#)). Simplicity is "nice".

12 · HOW TO DO IT

Build order

Phase 1

Supabase foundation. Create the project; run the schema (\$05); enable RLS + policies (\$06); set up Auth + the [reward-images](#) bucket; write the core RPCs.

Phase 2

Owner core. Owner signup/login, create café (unique-slug logic), dashboard shell — all mobile.

Phase 3

Diner + Ma carte. Phone+PIN auth, the loyalty card with balance + "almost there" nudge.

Phase 4

Caisse (Consume→Earn). Staff credits a phone; [credit_points](#) ; welcome bonus; the "+points" moment.

Phase 5

Games (the Spin). Wheel with server `play_game`, everyone-wins, cooldown, anticipation + celebration.

Phase 6

Boutique (the Reward). Catalogue + `redeem_at_counter` + code validation in the caisse.

Phase 7

Owner control panel. Every knob from \$09, grouped by moment.

Phase 8

Return layer. Streaks/daily, events, "your spin is ready", secondary meta-games (hideable).

Phase 9

Polish + mobile QA. Celebrations, empty states, error states — all at 375–430px.

13 · DEFINITION OF DONE

Acceptance checklist

- ☐ A diner scans a QR, signs up with phone+PIN in ~10s, and receives the welcome bonus.
- ☐ Staff credit points to a phone at the caisse; the diner's balance updates live with a "+points" moment.
- ☐ The wheel always returns a prize, respects the owner's cooldown, and cannot be re-triggered from the browser.
- ☐ A diner redeems a reward, gets a code, and staff can mark it used exactly once.
- ☐ "Almost there" progress to the next reward shows on Ma carte and the Boutique.
- ☐ The owner can change points rate, welcome bonus, rewards+prices, prizes+odds, cooldown, and events — and the client reflects it.
- ☐ Streaks advance on return; an active event multiplier is applied automatically.
- ☐ All value logic is server-side (RPC/service-role); RLS blocks cross-café reads/writes.
- ☐ **Every screen is built and tested at phone width (375–430px) only. No desktop layout exists.**

That's the build.

One loop — consume, earn, play, redeem, return — made fun, useful, and nice for the diner, and fully controlled by the owner. Supabase backend, server-authoritative, mobile-only. Ship it phase by phase.